OPERATING SYSTEMS LAB

PAGE REPLACEMENT ALGORITHM SIMULATOR

Project Report

Submitted by	Name: _____
Roll Number	_____
Course / Branch	_____
Submitted to	_____
Institution	_____
Academic Year	2026

Programming Language: Python

Environment: Google Colab / Jupyter Notebook

Abstract

This project implements a Page Replacement Algorithm Simulator for the Operating Systems laboratory. It demonstrates three page replacement policies: First-In, First-Out (FIFO), Least Recently Used (LRU), and Optimal. The program accepts a page reference string and a frame count, records the frame contents after each page reference, identifies hits and faults, and calculates hit and fault ratios. A final comparison table helps observe how the policies behave on the same input. The simulator is intended for learning and experimentation; results depend on the reference string and number of frames.

1. Introduction

Virtual memory allows an operating system to run processes without requiring every page of a process to remain in physical memory at all times. When a referenced page is not currently loaded, a page fault occurs. If free frames are available, the page can be loaded into one. If all frames are occupied, the operating system must select a page to remove. A page replacement algorithm defines how that victim page is chosen.

This project provides a practical demonstration of three common replacement algorithms and reports the number of page faults, page hits, hit ratio, and fault ratio.

2. Objectives

• Understand page faults, page hits, and page replacement in virtual memory.

• Implement FIFO, LRU, and Optimal page replacement algorithms in Python.

• Accept and validate the page reference string and number of frames at runtime.

• Display a trace containing the step, referenced page, frame contents, result, and replaced page.

• Calculate page faults, page hits, hit ratio, and fault ratio.

• Compare all three algorithms and investigate Belady's anomaly.

3. Problem Statement and Scope

The task is to simulate FIFO, LRU, and Optimal using the same input. The frame set begins empty. For each page reference, the program determines whether the reference is a hit or a fault, updates frames if necessary, and records the operation. Optimal uses future references to select the page whose next use is farthest away. If multiple pages are never used again, the implementation follows the assignment rule and replaces the page appearing first in the frame list.

4. Algorithm Overview

4.1 FIFO — First-In, First-Out

FIFO replaces the page that has been resident in memory for the longest time. The implementation maintains a rotating replacement position, similar to a queue. FIFO is straightforward to implement, but it does not consider how recently or how frequently a page was used.

4.2 LRU — Least Recently Used

LRU replaces the page whose most recent reference occurred furthest in the past. The simulator records the last-used step for each page currently in memory. LRU uses recent history as an estimate of near-future use, but it requires additional bookkeeping.

4.3 Optimal

Optimal replaces the resident page whose next reference is farthest in the future; if a page will not be referenced again, it is a candidate for replacement. Under the same reference string and frame count, Optimal provides the

minimum possible number of page faults among these policies. It is mainly a benchmark because a real operating system cannot know future references exactly.

5. System Design and Workflow

Input reference string and frame count → validate input → run FIFO, LRU, and Optimal independently → record each step → calculate metrics → display comparison.

• Input module: parses pages separated by spaces or commas and checks that the frame count is a positive integer.

• Algorithm modules: maintain frames and choose a victim page according to each policy.

• Trace module: stores step number, page, frame snapshot, hit/fault status, and replaced page.

• Metrics module: calculates total hits, total faults, hit ratio, and fault ratio.

• Comparison module: prints the results for all three algorithms in one table.

6. Performance Metrics

Metric	Definition
Page hit	A referenced page is already present in a frame.
Page fault	A referenced page is not present in any frame.
Hit ratio	Number of hits ÷ total page references.
Fault ratio	Number of faults ÷ total page references.

The ratios can be displayed as percentages by multiplying each ratio by 100. For a non-empty reference string, hits plus faults equals the total number of references, and hit ratio plus fault ratio equals 1 (or 100%).

7. Sample Input and Expected Metrics

The assignment provides this sample: reference string = 1 2 3 1 4 2 5, number of frames = 3. The assignment's sample run reports FIFO: 5 faults and 2 hits; LRU: 6 faults and 1 hit; Optimal: 5 faults and 2 hits. These figures are included here as the assignment's stated sample output. Run the submitted code to verify actual outputs before final submission.

Algorithm	Faults	Hits	Hit Ratio
FIFO	5	2	28.57%
LRU	6	1	14.29%
Optimal	5	2	28.57%

8. Experiments and Analysis

8.1 Belady's Anomaly

Belady's anomaly is a phenomenon in which increasing the number of frames causes a page replacement algorithm to produce more page faults. FIFO can exhibit this anomaly. LRU and Optimal have the stack property and do not exhibit this behaviour under the standard model.

Use the assignment reference string 1 2 3 4 1 2 5 1 2 3 4 5 with 3 frames and then 4 frames. Record the actual fault counts from the executed simulator in the table below.

Algorithm	Faults (3 frames)	Faults (4 frames)
FIFO	_____	_____
LRU	_____	_____
Optimal	_____	_____

Question: Does any algorithm produce more faults with more frames? Identify the algorithm from the observed results and explain why LRU and Optimal do not show this behaviour.

8.2 Custom Reference Strings

The assignment also asks for two reference strings of at least 15 pages: one where LRU performs clearly better than FIFO, and one where FIFO is better than or equal to LRU. Use the simulator to test candidate strings and enter the verified values below.

Case	Reference string	Frames	FIFO faults	LRU faults
LRU better	_____	___	___	___
FIFO better/equal	_____	___	___	___

9. Short Questions

Q1. Why can Optimal not be implemented exactly in a real operating system?

Optimal requires knowledge of future page references. Since an operating system cannot know a process's exact future memory-access sequence, Optimal is used as a theoretical benchmark rather than a practical exact policy.

Q2. What extra hardware or bookkeeping does LRU need compared with FIFO?

LRU must track how recently each page was used, using timestamps, counters, a stack, or another data structure. FIFO primarily needs to maintain the order in which pages entered memory.

Q3. If the number of frames equals the number of distinct pages in the string, how many faults occur?

If all frames start empty and there are enough frames to hold every distinct page, each distinct page faults the first time it is referenced. It is then kept in memory, so later references to it are hits. Therefore, total faults equal the number of distinct pages.

10. Conclusion

The Page Replacement Algorithm Simulator provides a hands-on way to understand how FIFO, LRU, and Optimal manage a limited number of memory frames. Step-by-step traces make the victim-selection rules visible, while the performance metrics provide a consistent basis for comparison. FIFO is simple but can exhibit Belady's anomaly; LRU uses past access history; Optimal uses future knowledge and acts as a benchmark. Results should be verified by executing the notebook with the required assignment inputs.

11. How to Run

• Open the .ipynb notebook in Google Colab or Jupyter Notebook.

• Run cells from top to bottom.

• Enter a numeric page reference string separated by spaces or commas.

• Enter a positive integer for the number of frames.

• Review the trace tables, fault/hit totals, ratios, and final comparison.

• Run the manual-tracing and experiment cells; fill the tables in this report using verified outputs.

12. References

• Operating Systems Lab Assignment: Page Replacement Algorithm Simulator (provided assignment handout).

• Course notes and instructor-provided materials on virtual memory and page replacement.